

bracIIT

BRAC IT CodeSprint 2026 — Project BRD

1. **Project Name:** DFA Support Copilot — AI-assisted root cause analysis for L1/L2 support
2. **Project Owner:** Md Ershadul Hoque Sarker · Employee PIN 2398

3. Problem Statement

- A DFA transaction crosses three systems — **Android app → Mission Control → client CBS/T24** — but a support ticket usually arrives with only a symptom: “collection failed”, “stuck in syncing”, a transaction ID.
- Diagnosing it means manually querying PostgreSQL and MongoDB logs across those hops, correlating timestamps, and interpreting CBS error codes. Most L1 agents cannot do this.
- The volume is real.** In Tanzania alone, ~80,316 of 2.31M app-facing calls failed in a two-week window. Roughly 39,809 Tanzania calls carried no valid latency log at all, so even the timing evidence is incomplete.
- The knowledge is concentrated.** Accurate diagnosis depends on a handful of experienced engineers who know the CBS error vocabulary and where to look. When they are unavailable, tickets wait.
- The result is over-escalation.** Because L1 cannot confidently classify an issue as configuration, data, application or infrastructure, tickets escalate to L2/L3/Development by default — including issues needing only a data correction, and issues that were never DFA’s to fix (MC → CBS runs at 99.99%).
- The gap in one line:** the evidence needed to resolve most DFA tickets already exists in the logs, but it is not reachable by the person who receives the ticket.

4. Proposed Solution

A **standalone web application** for L1/L2 support agents. No changes to the DFA codebase; **read-only** access to the UAT/dev PostgreSQL and MongoDB instances via **MCP (Model Context Protocol)** servers.

How it works

- Agent enters an error message, transaction ID, member ID, or a plain-language description of the problem.
- The app resolves the input to the relevant records and pulls the correlated log trail across the three hops — app request, Mission Control processing, queue handoff, CBS response.
- An LLM analyses that evidence against a curated SOP/runbook knowledge base and returns a structured diagnosis.
- The agent reviews, validates, and acts.

Output for every query

Field	Example
Category	CBS integration / data issue
Probable root cause	Invalid CBS account status
Where it stopped	CBS/T24 — Mission Control delivered successfully
Owner	Client CBS — not a DFA application defect
Recommended steps	Verify CBS response code and account status
Data correction needed	Yes
Application bug indicated	No
Escalation required	L2 only if CBS data is valid
Related SOP	CBS Loan Collection Rejection SOP
Similar past incidents	3 matches, with resolutions
Confidence	High / Medium / Low

Two non-negotiable design rules

- Advisory only.** The Copilot recommends; the L1/L2 agent validates and acts. It never modifies data or triggers actions. Accountability stays with the human.
- Evidence-linked.** Every claim cites the specific log record it came from, expandable to the raw row. If the logs do not support a conclusion, the Copilot returns “insufficient evidence” rather than a guess.

5. Target Users

- Primary:** L1 and L2 support agents handling DFA tickets — the people who receive the issue first and currently lack the tooling to diagnose it.
- Secondary:** L3 / development engineers, who receive fewer and better-qualified escalations, each arriving with evidence already attached.
- Benefiting indirectly:** field officers and client institutions through faster resolution; BRAC IT through reduced dependency on a small number of experienced engineers.

6. Key Features

- Natural-language and ID-based intake** — error text, transaction ID, member ID, or a description.
- Cross-hop log correlation** — assembles the app → MC → CBS trail for a single transaction from PostgreSQL and MongoDB.
- Root cause analysis** with category, probable cause, and confidence level.
- Classification** into configuration / data / application / infrastructure, with ownership assigned between DFA and the client CBS.
- Recommended troubleshooting steps**, ordered, specific to the diagnosed cause.
- SOP / runbook retrieval** — surfaces the relevant procedure alongside the diagnosis.
- Escalation guidance** — whether L2/L3/Development involvement is warranted, and why.
- Similar incident lookup** — past tickets with matching signatures and how they were resolved.
- Evidence panel** — the raw log records behind every conclusion.
- Feedback capture** — agents mark each diagnosis correct or incorrect, building an evaluation set for future tuning.

Anchor scenario for the build and demo: **Tanzania**.

7. AI Component

Where	What the AI does
Log retrieval	MCP servers expose read-only PostgreSQL and MongoDB access. The model composes queries to gather evidence rather than relying on pre-built reports.
Root cause reasoning	Interprets the correlated log trail and CBS error codes to produce a probable cause.
Classification	Assigns category, ownership and escalation recommendation from a fixed schema — structured JSON output, not free text.
SOP retrieval	Semantic search over the runbook knowledge base to attach the right procedure.
Incident matching	Finds historically similar failures by signature and surfaces their resolutions.
Natural language interface	Lets an agent describe a problem in their own words instead of knowing which table to query.

Where AI is deliberately *not* used: transaction states, timestamps, status codes and counts are read directly from the logs and displayed as-is. The model interprets evidence; it never generates a fact about the system. This keeps every output traceable to a source record.

Model choice is open — Claude, GPT, or an on-prem model. The MCP layer and the output schema are model-agnostic, so the model can be swapped if data-residency requirements demand an internal deployment.

8. Expected Outcome / Impact

Dimension	Today	With the Copilot
L1 first-response diagnosis	Rarely possible	Structured diagnosis in under a minute
Log correlation across three hops	Manual, senior-engineer skill	Automatic
Escalation decision	Default to escalate	Evidence-based recommendation
Escalations reaching L2/L3	High proportion avoidable	Reduced — target measured in pilot
Knowledge dependency	Concentrated in a few engineers	Encoded in SOPs and reusable
Escalation quality	Symptom only	Evidence bundle attached

Strategic value: DFA-side integration is already reliable (MC → CBS at 99.99%). Most tickets are data issues or client-CBS faults. A tool that establishes *which* — quickly and with evidence — converts support effort into resolution instead of investigation, and gives BRAC IT a defensible position when the fault sits with a client’s core banking vendor.

9. Data / Resources Required

- **Systems (read-only):** PostgreSQL (UAT/dev) and MongoDB (UAT/dev), both via MCP server; anonymised real integration logs — confirmed available — for build and evaluation.
- **Content:** 10–15 existing SOPs / runbooks for common DFA failure scenarios; a set of historical resolved tickets for incident matching and accuracy testing; CBS/T24 error code reference for the anchor country.
- **Technical:** LLM API access or an on-prem model endpoint; one environment to host the standalone app (a laptop is sufficient for the hackathon).
- **Explicitly not required:** DFA codebase access, write access to any environment, Android app changes, production data.

10. Success Criteria

Functional

- Given a transaction ID from the anonymised dataset, the Copilot returns a complete structured diagnosis in under 60 seconds.
- Every conclusion links to the log records supporting it.
- Queries with insufficient evidence return “insufficient evidence”, not a fabricated cause.

Accuracy — measured against 30–50 historical tickets with known resolutions

- Category classification correct in ≥80% of cases.
- Escalation recommendation matches the actual outcome in ≥75% of cases.
- **Zero fabricated facts** — no invented transaction IDs, statuses or timestamps. A pass/fail gate, not a percentage.

Usability

- An L1 agent with no database knowledge can operate it unaided.
- The correct SOP is surfaced for the top failure scenarios.

Pilot measure (post-hackathon): reduction in average L1 resolution time and in the proportion of tickets escalated to L2/L3.

Why Should You Build This?

Every DFA support ticket today starts with a person who can see the symptom but not the cause, and ends — too often — with an escalation that was never needed. The evidence to resolve most of these tickets already sits in our PostgreSQL and MongoDB logs; it is simply out of reach of the person holding the ticket. This project puts it in reach, with a diagnosis, an SOP and an evidence trail attached. It is buildable in the round: standalone, read-only, no DFA codebase dependency, MCP does the data access, and we already have anonymised logs to build and test against. And it is measurable — we can score it against real historical tickets, not opinions.

Appendix — Build scope for the round

Constraint: 3 people, ~20 hours each alongside daily work.

In scope: MCP read-only connectors; log correlation for the top 3 Tanzania failure scenarios; diagnosis engine with structured output; SOP retrieval over 10–15 documents; single-page web UI; evidence panel; accuracy evaluation against a labelled ticket set.

Deferred: similar-incident matching at scale; multi-country configuration; feedback-driven tuning; production hardening; integration into existing support tooling.

Suggested split: one engineer on MCP and data access, one on the diagnosis engine and prompts, one on UI and the evaluation harness.